

# Open Source Prototype Systems

## Week 14 Student Handout

### One-Hour RTL Exercise: Security in a Toy Accelerator

Topic: Timing leakage, fixed-latency RTL, debug-interface discipline, and open-source verification tools

## 1. Exercise Overview

- **Exercise goal:** Reproduce and mitigate a timing side channel in a small RTL password-token comparator
- **Core security property:** Different secret-dependent mismatch positions should not change software-visible completion timing
- **RTL target:** A toy accelerator interface with `clk`, `rst`, `start`, `candidate`, `done`, `match`, and `debug_count` signals
- **Open-source tools:** Verilator for simulation, GTKWave for traces, Yosys for synthesis estimates, and Python for latency analysis
- **Expected outcome:** A fixed RTL implementation that preserves functional correctness while making latency and public debug output constant
- **Course connection:** The exercise connects Week 11 verification, Week 12 custom accelerators, Week 13 power measurement, and Week 14 security review

## 2. Learning Outcomes

- **Threat-model interpretation:** Identify which attacker observation makes a timing leak relevant in a prototype system
- **Microarchitectural leakage:** Explain how an early-exit control path leaks information even when final outputs look correct
- **Constant-time RTL:** Implement fixed-latency comparison using an accumulated mismatch flag rather than early termination
- **Debug-interface safety:** Recognize why counters and internal progress signals can become software-visible side channels
- **Trace-based debugging:** Use waveforms to connect security symptoms to state transitions and control decisions
- **Trade-off measurement:** Compare latency, approximate area, and activity implications before and after mitigation

## 3. Suggested One-Hour Plan

| Time      | Activity                                                                                 | Student checkpoint                                |
|-----------|------------------------------------------------------------------------------------------|---------------------------------------------------|
| 0-10 min  | Read threat model, inspect RTL interface, build the buggy design                         | Can explain which signal leaks timing             |
| 10-25 min | Run buggy simulation, inspect CSV and waveform, map mismatch byte to latency             | Can show the leak using a table or VCD            |
| 25-45 min | Modify <code>secure_compare_fixed.sv</code> to remove early exit and hide debug progress | Fixed design compiles and passes functional cases |
| 45-55 min | Run constant-latency check, generate plots or printed summaries, compare traces          | All input classes complete in one latency         |
| 55-60 min | Record area/latency/security trade-off and prepare submission files                      | Short explanation ready for TA review             |

## 4. Threat Model and Security Property

- **Asset:** The secret token stored inside the accelerator is confidential
- **Attacker capability:** Untrusted software can submit candidates and measure when done becomes visible
- **Allowed output:** The final match bit may reveal whether the full candidate equals the secret
- **Forbidden output:** The byte position of the first mismatch must not be inferable from timing or public counters
- **Observation channel:** Completion latency and debug\_count are treated as software-visible outputs
- **Security target:** For the same public request type, observable traces should be independent of secret-dependent mismatch position

## 5. Starter Package Structure

| File or directory                 | Purpose                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------|
| rtl/secure_compare_buggy.sv       | Deliberately insecure design used to reproduce the side channel                               |
| rtl/secure_compare_fixed.sv       | Student-editable RTL skeleton where the mitigation is implemented                             |
| tb/tb_secure_compare.cpp          | Verilator C++ testbench that runs match, mismatch, and random candidate cases                 |
| scripts/check_constant_latency.py | Regression check for constant latency, correct match behavior, and stable public debug output |
| scripts/analyze_latency.py        | Text and optional plot generator for latency distributions                                    |
| scripts/count_vcd_transitions.py  | Simple activity proxy based on VCD value-change records                                       |
| Makefile                          | Unified build, simulation, waveform, synthesis, and cleanup commands                          |
| Dockerfile and run_docker.sh      | Optional container workflow based on the course Docker environment                            |

## 6. Setup and Basic Commands

- **Native setup:** Use a Linux environment with Verilator, Yosys, GTKWave, Make, g++, Python 3, and optionally matplotlib
- **Docker setup:** Use the `amansinhaatnycu/osp-week14-security-exercise:latest` docker image

```
# Change to starter package folder and start docker container
```

```
cd week14_Starter/
```

```
docker run --rm -it \
```

```
-v "$(pwd)":/work \
```

```
-w /work \
```

```
amansinhaatnycu/osp-week14-security-exercise:latest \
```

```
bash
```

```
# Inside the container or native Linux environment
```

```
make sim-buggy
```

```
make sim-fixed
```

```
make test
```

```
make analyze
```

```
make synth
```

- **Buggy expectation:** make sim-buggy should show different completion latencies for different mismatch positions
- **Initial fixed expectation:** make test is expected to fail until secure\_compare\_fixed.sv is corrected
- **Waveform command:** Use make wave-buggy or make wave-fixed to inspect generated VCD traces in GTKWave

## 7. Part A: Reproduce the Leak

- **Run simulation:** Execute `make sim-buggy` and inspect `build/latency_buggy.csv`
- **Compare candidates:** Check exact match, `mismatch_byte0`, `mismatch_byte1`, `mismatch_byte3`, `mismatch_byte7`, and random cases
- **Build timing table:** Record how many cycles each named case takes before done is asserted
- **Open waveform:** Inspect start, done, match, idx, running, and `debug_count` in `build/trace_buggy.vcd`
- **Locate root cause:** Find the branch that terminates execution when `byte_equal` becomes false
- **Reason about visibility:** Explain why match alone is safe but completion timing and `debug_count` are not safe

## 8. Part B: Implement the Mitigation

- **Remove early exit:** Continue processing every byte even after the first mismatch is detected
- **Accumulate mismatch:** Add a register such as `mismatch_seen` that records whether any byte has mismatched
- **Fix done timing:** Assert done only after the final byte or a fixed final cycle count
- **Compute match safely:** Set match to one only when no mismatch has been accumulated after all bytes are scanned
- **Hide progress counter:** Keep public `debug_count` constant or disable it for the untrusted interface
- **Avoid random delay:** Do not use randomness as the primary fix because it weakens reproducibility and formal reasoning
- **Preserve interface:** Do not change the top-level port names or widths because the testbench and build scripts depend on them

## 9. Part C: Verify the Fix

```
make sim-fixed
make test
make analyze
make wave-fixed
```

- **Functional correctness:** Exact match should return `match=1` and all mismatches should return `match=0`
- **Latency equality:** All candidate classes should report the same latency in `build/latency_fixed.csv`
- **Debug stability:** Public `debug_count` should not reveal byte index or mismatch position
- **Trace comparison:** The fixed waveform should keep scanning after a byte mismatch and assert done at the final fixed point
- **Regression habit:** Rerun `make test` after every RTL edit to avoid a functional fix that reintroduces leakage

## 10. Part D: Measure Trade-Offs

```
make synth
make transitions
python3 scripts/analyze_latency.py build/latency_buggy.csv build/latency_fixed.csv
```

- **Latency cost:** The fixed design removes fast mismatch cases, so best-case latency increases intentionally
- **Area cost:** Yosys stat reports reveal extra registers or logic from the mismatch accumulator and fixed control path
- **Activity proxy:** VCD transition counts provide a rough switching-activity comparison, not a replacement for power analysis
- **Security gain:** The mismatch-position timing channel is removed from done timing and public `debug_count`
- **Residual channels:** Power, EM behavior, synthesis optimization, and privileged debug may still require deeper review

- **Design judgment:** A reusable open-source accelerator should document both the mitigation and its measurable overhead

## 11. Required Submission

- **Patched RTL:** Submit the final version or patch for rtl/secure\_compare\_fixed.sv
- **Timing evidence:** Submit a small table comparing buggy and fixed latencies for at least four input classes
- **Waveform evidence:** Submit one screenshot showing start, done, match, idx or equivalent state, and debug\_count
- **Tool evidence:** Submit the relevant make command outputs or generated CSV files
- **Trade-off explanation:** Write five to eight sentences explaining the security property, overhead, and remaining limitations

## 12. Suggested Grading Rubric

| Item                  | Points | Criteria                                                                    |
|-----------------------|--------|-----------------------------------------------------------------------------|
| Leak reproduction     | 20     | Correctly identifies and measures the buggy timing leak                     |
| RTL mitigation        | 30     | Removes early exit, preserves correctness, and fixes done timing            |
| Debug policy          | 15     | Prevents public debug_count from exposing internal progress                 |
| Verification evidence | 20     | Provides CSV, waveform, and constant-latency test results                   |
| Trade-off explanation | 15     | Clearly discusses latency, area, activity, and residual-channel limitations |

## 13. Troubleshooting Checklist

- **Verilator missing:** Use the Docker workflow or install verilator through the system package manager
- **GTKWave unavailable:** Open the VCD later on another machine or submit CSV evidence if GUI access is unavailable
- **Test still fails:** Check whether done timing, match computation, or public debug\_count still varies across cases
- **Functional mismatch:** Verify that mismatch\_seen is cleared at start and updated before the final match decision
- **Synthesis warning:** Read Yosys messages carefully, especially warnings about unused signals or optimized-away logic
- **Ambiguous result:** Use waveform inspection to confirm which cycle contains start, each byte comparison, and done